

Machine Learning techniques for finding Legendre pairs

July 2026

Abstract

This document details how different machine learning techniques may be applied in the search for Legendre pairs. For the training of any supervised machine learning models, we assume we have an exhaustive list of known Legendre pairs for $n \leq 57$.

1 Classifier for m -compressions

1.1 Motivation

The compression-decompression approach for finding Legendre pairs consists in looking for a suitable compression of a (u, v) pair. If $u \in \{-1, 1\}^n$, where $n = m \cdot p$, we can write the m -compression $U^{(m)}$ of u as

$$U^{(m)} = [u_0^{(m)}, \dots, u_p^{(m)}] \quad , \text{ where } \quad u_i^{(m)} = \sum_{j=0}^{m-1} u_{i+mj} . \quad (1)$$

A preliminary test may be performed on $U^{(m)}$ to check if can be the compression of a vector that forms a Legendre pair, the *individual PSD test*. This test states that, if (u, v) form a Legendre pair and $U^{(m)}$ is the m -compression of u , then

$$PSD(U^{(m)}, s) \leq 2n + 2 \quad \forall s \in \{1, \dots, m-1\} . \quad (2)$$

This test allows us to very efficiently reduce the search space for valid pairs. Now, given any two compressed vectors $(U^{(m)}, V^{(m)})$, we can check if they may be the compressions of a Legendre pair through the *joint PSD test*, which states that, if $(U^{(m)}, V^{(m)})$ are the compressions of a Legendre pair (u, v) , then

$$PSD(U^{(m)}, s) + PSD(V^{(m)}, s) = 2n + 2 \quad \forall s \in \{1, \dots, m-1\} . \quad (3)$$

In this document, we shall call a $(U^{(m)}, V^{(m)})$ pair *viable* if it satisfies the individual and joint PSD tests. We call a $(U^{(m)}, V^{(m)})$ pair *good* if it is the compression of a Legendre pair.

Even though this approach allows us to discard many pairs, the decomposition of a viable $(U^{(m)}, V^{(m)})$ pair (i.e. finding a Legendre pair that has $(U^{(m)}, V^{(m)})$ as its compression, if one exists) is still a costly process, due to the fact that a factorial number of vectors may share the same compression.

One possible way to use machine learning techniques to make this search more efficient is to train a model that predicts the probability (in a Bayesian sense) that a given viable compressed pair can be decompressed to a Legendre pair. If such a model is available, then we can order the set of viable pairs based on the probability of them being good. Then, if a Legendre pair of length n exists, we should be able to find it by performing exhaustive search on the highest-ranking compressed pairs.

1.2 Dataset

Legendre pairs are known to be preserved by shifting and order reversion of either vector, and to decimation of both vectors together (charm-bracelet equivalence class). When performing any sort of brute forcing, this invariance should be utilized to reduce the search space, for instance by representing the vectors in some canonical form that is equal for two pairs if and only if they are equivalent. Let us call this canonical form $C(u, v)$.

One way of ensuring the neural model respects this invariance is to condition the input so that the model always receives $C(u, v)$. The computational cost for computing $C(u, v)$ for each pair in the training dataset should be relatively small compared to the training time.

Another approach is to use the invariance as a form of data augmentation, which would balance the fact that the available datasets are small, and allow the model itself to learn to satisfy the equivalence classes from the data. This may lead to more robustness when we evaluate the model on values of n that weren't in the training data.

Yet another possibility is to train an embedding to learn the charm-bracelet invariance, that is, an embedding that's trained to produce the same result for e.g. different shifts of the same vector. This can be achieved through self-supervised learning, which doesn't require any labeled data, using techniques such as Variance-Invariance-Covariance regularization [1, 2].

1.3 Architecture

There are essentially three levels to how general one can make the model: it can be trained for fixed m, p ; for fixed p but possibly varying m ; or for any n . Each choice will have consequences in terms of the neural network's architecture.

Fixed n . If we fix m, p , then the network can be relatively simple, as the model will only need to judge compressed pairs of vectors of length p , which are the m -compressions of vectors of length n , outputting the probability that the pair is good. This can be achieved by a Multi-Layer Perceptron (MLP) with input size $2p$ that receives the concatenation of $U^{(m)}$ and $V^{(m)}$ and produces

an output with sigmoid activation function, ensuring it's between 0 and 1. The clear limitation of this approach is that it cannot be used for finding Legendre pairs of new lengths, such as $n = 115$, which are the cases we are most interested in.

Fixed p . In this case, we can still use an MLP with fixed size, however it becomes necessary to pass m as an additional input to the model, as the same pair of vectors of length p could be e.g. a good 3-compression but a bad 5-compression. By training such a model for a small value of p , we can then apply to greater values of n that are multiples of p . As an example, we could train for $p = 5$ using the available data for $n = 5, 10, \dots, 55$ and then use this model to tackle the open problem $n = 115$.

General n . The more general case requires more complex architectures than the MLP, since now the vector length p can be any natural number. Recurrent Neural Networks (RNNs) or variations such as Long-Short-Time Memory (LSTM) could be used to read the pair $U^{(m)}, V^{(m)}$ in order from beginning to end and finally output the desired probability. Alternatively, we could use transformers, taking advantage of the fact that the input is already discrete ($u_i^{(m)}$ is always an integer), and thus there is a natural tokenization. Transformers tend to require larger amounts of data than MLPs, however this may be offset by the fact that a model for general n will have a much wider dataset than the previously mentioned restricted models.

References

- [1] Adrien Bardes, Jean Ponce, and Yann LeCun. “Vicreg: Variance-invariance-covariance regularization for self-supervised learning”. In: *arXiv preprint arXiv:2105.04906* (2021).
- [2] Ravid Shwartz-Ziv et al. “An information theory perspective on variance-invariance-covariance regularization”. In: *Advances in neural information processing systems* 36 (2023), pp. 33965–33998.